

P12 Operational Playbooks for WatchLockAI Sentinel

Generated: 2025-09-06T23:45:00Z

Sprint: P12 - Operational Playbooks & Diff Bundles

Scope: Production operations, incident response, and maintenance procedures

Executive Summary

Comprehensive operational playbooks for WatchLockAI Sentinel covering all aspects of production deployment, monitoring, incident response, and maintenance. These playbooks provide step-by-step procedures for operations teams to ensure reliable, secure, and performant operation of the Sentinel system.

Playbook Categories:

- **Deployment & Installation:** Production deployment procedures
- **Monitoring & Alerting:** Operational monitoring and alert response
- **Incident Response:** Security incident handling and escalation
- **Maintenance & Updates:** System maintenance and upgrade procedures
- **Troubleshooting:** Diagnostic and resolution procedures
- **Security Operations:** Security-focused operational procedures

1. Deployment & Installation Playbooks

1.1 Production Deployment Checklist

Pre-Deployment Verification

```
# System requirements check
./scripts/preflight_check.sh

# Environment variables validation
python tools/verify_minimax_claims.py --check-env

# Security configuration audit
python tools/sec_lint.py --production-mode .

# Performance baseline establishment
python tools/microbench.py --baseline --output
baseline_pre_deploy.json
```

Production Deployment Steps

1. Environment Preparation

```
```bash
Create service user
sudo useradd -r -s /bin/false sentinel-service

Create application directories
sudo mkdir -p /opt/sentinel/{app,data,logs,config}
sudo chown -R sentinel-service:sentinel-service /opt/sentinel

Set appropriate permissions
sudo chmod 750 /opt/sentinel/app
sudo chmod 700 /opt/sentinel/data
```

```
sudo chmod 755 /opt/sentinel/logs
```\
```

1. Application Installation

```
```\bash
Extract application package
cd /opt/sentinel/app
sudo -u sentinel-service unzip watchlockai_sentinel-*.zip

Install dependencies
sudo -u sentinel-service pip install -r requirements.txt

Set up configuration
sudo cp config/production.yaml /opt/sentinel/config/
sudo chown sentinel-service:sentinel-service /opt/sentinel/config/production.yaml
sudo chmod 600 /opt/sentinel/config/production.yaml
```\
```

1. Service Configuration

```
```\bash
Install systemd service
sudo cp scripts/sentinel_service_runner.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable sentinel_service_runner

Configure log rotation
sudo cp config/sentinel-logrotate /etc/logrotate.d/sentinel
```\
```

1. Security Hardening

```
```\bash
Configure firewall
sudo ufw allow 8080/tcp comment "Sentinel API"
sudo ufw reload

Set SELinux contexts (if applicable)
sudo setsebool -P httpd_can_network_connect 1

Configure SSL certificates
sudo cp ssl/sentinel.crt /opt/sentinel/config/
sudo cp ssl/sentinel.key /opt/sentinel/config/
```

```
sudo chmod 400 /opt/sentinel/config/sentinel.key
...
```

## Post-Deployment Validation

```
Start service
sudo systemctl start sentinel_service_runner

Verify service status
sudo systemctl status sentinel_service_runner

Health check validation
curl -f http://localhost:8080/health || echo "Health check failed"

API functionality test
python tools/sentinel_sdk.py http://localhost:8080

Performance validation
python tools/microbench.py --validate baseline_pre_deploy.json
```

## **1.2 Container Deployment (Docker)**

### **Production Docker Deployment**

```
docker-compose.prod.yml
version: '3.8'
services:
 sentinel:
 image: watchlockai/sentinel:latest
 restart: unless-stopped
 ports:
 - "8080:8080"
 environment:
 - SENTINEL_ENV=production
 - HEALTH_ENDPOINT_ENABLED=1
 - ADMIN_AUTH_ENABLED=1
 - RATE_LIMIT_ENABLED=1
 volumes:
 - ./data:/app/data:rw
 - ./logs:/app/logs:rw
 - ./config:/app/config:ro
 healthcheck:
 test: ["CMD", "curl", "-f", "http://localhost:8080/health"]
 interval: 30s
 timeout: 10s
 retries: 3
 start_period: 60s
 deploy:
 resources:
 limits:
 memory: 2G
 cpus: '1.0'
 reservations:
 memory: 1G
 cpus: '0.5'
```

## Container Deployment Commands

```
Deploy to production
```

```
docker-compose -f docker-compose.prod.yml up -d
```

```
Verify deployment
```

```
docker-compose -f docker-compose.prod.yml ps
```

```
docker-compose -f docker-compose.prod.yml logs sentinel
```

```
Health check
```

```
docker exec sentinel_sentinel_1 curl -f http://localhost:8080/
health
```

## **2. Monitoring & Alerting Playbooks**

### **2.1 Health Monitoring Setup**

#### **Continuous Health Monitoring Script**

```
#!/bin/bash
sentinel_health_monitor.sh - Production health monitoring

SENTINEL_URL="http://localhost:8080"
CHECK_INTERVAL=30
LOG_FILE="/var/log/sentinel/health_monitor.log"
ALERT_EMAIL="ops-team@example.com"

monitor_health() {
 timestamp=$(date -Iseconds)

 # Health check
 if curl -sf "${SENTINEL_URL}/health" > /dev/null; then
 echo "<span class='math-inline' style='display:
inline;'><math xmlns='http://www.w3.org/1998/Math/MathML'
display='inline'><mrow><mrow><mi>t</mi><mi>i</mi><mi>m</mi><mi>e</
mi><mi>s</mi><mi>t</mi><mi>a</mi><mi>m</mi><mi>p</mi></mrow><mo
stretchy='false'>[</mo><mi>I</mi><mi>N</mi><mi>F</mi><mi>O</mi><mo
stretchy='false'>]</mo><mi>H</mi><mi>e</mi><mi>a</mi><mi>l</
mi><mi>t</mi><mi>h</mi><mi>c</mi><mi>h</mi><mi>e</mi><mi>c</
mi><mi>k</mi><mi>:</mi><mi>O</mi><mi>K</mi><mi>"</
mi><mo>></mo><mo>></mo><mi>"</mi></mrow></math></
span>LOG_FILE"
 return 0
 else
 echo "<span class='math-inline' style='display:
inline;'><math xmlns='http://www.w3.org/1998/Math/MathML'
display='inline'><mrow><mrow><mi>t</mi><mi>i</mi><mi>m</mi><mi>e</
mi><mi>s</mi><mi>t</mi><mi>a</mi><mi>m</mi><mi>p</mi></mrow><mo
stretchy='false'>[</mo><mi>E</mi><mi>R</mi><mi>R</mi><mi>O</
mi><mi>R</mi><mo stretchy='false'>]</mo><mi>H</mi><mi>e</mi><mi>a</
mi><mi>l</mi><mi>t</mi><mi>h</mi><mi>c</mi><mi>h</mi><mi>e</
mi><mi>c</mi><mi>k</mi><mi>:</mi><mi>F</mi><mi>A</mi><mi>I</
mi><mi>L</mi><mi>E</mi><mi>D</mi><mi>"</mi><mo>></
mo><mo>></mo><mi>"</mi></mrow></math>LOG_FILE"
```

```
Send alert
echo "Sentinel health check failed at ${timestamp}" | \
 mail -s "ALERT: Sentinel Health Check Failed"
"$ALERT_EMAIL"

 return 1
fi
}

Main monitoring loop
while true; do
 monitor_health
 sleep "$CHECK_INTERVAL"
done
```

## Prometheus Integration

```
prometheus_sentinel.yml
global:
 scrape_interval: 15s

scrape_configs:
 - job_name: 'sentinel'
 static_configs:
 - targets: ['localhost:8080']
 metrics_path: '/api/metrics/prometheus'
 scrape_interval: 30s

rule_files:
 - "sentinel_alerts.yml"

alerting:
 alertmanagers:
 - static_configs:
 - targets:
 - alertmanager:9093
```

## Grafana Dashboard Configuration

```
{
 "dashboard": {
 "title": "WatchLockAI Sentinel Operations",
 "panels": [
 {
 "title": "Service Health",
 "type": "stat",
 "targets": [
 {
 "expr": "sentinel_health_status",
 "legendFormat": "Health Status"
 }
]
 },
 {
 "title": "Event Bus Metrics",
 "type": "graph",
 "targets": [
 {
 "expr": "sentinel_event_bus_delivery_success_total",
 "legendFormat": "Successful Deliveries"
 },
 {
 "expr": "sentinel_event_bus_delivery_failure_total",
 "legendFormat": "Failed Deliveries"
 }
]
 }
]
 }
}
```

## 2.2 Alert Response Procedures

### Critical Alert Response (P1)

**Trigger:** Service down, critical security incident, data corruption

#### Immediate Response (0-15 minutes):

1. Acknowledge alert in monitoring system
2. Check service status: `sudo systemctl status sentinel_service_runner`
3. Review recent logs: `sudo tail -100 /var/log/sentinel/app.log`
4. Attempt service restart if applicable: `sudo systemctl restart sentinel_service_runner`
5. Notify incident commander and escalate to engineering team

#### Investigation Phase (15-60 minutes):

1. Collect diagnostic information:

```
```bash
```

```
# System diagnostics
```

```
python tools/composite_health_check.py --full-diagnostic
```

```
# Performance analysis
```

```
python tools/microbench.py --diagnostic-mode
```

```
# Security scan
```

```
python tools/sec_lint.py --incident-mode
```

```
```
```

1. Check for security indicators:

```
```bash
```

```
# Check for security events
```

```
python tools/verify_minimax_claims.py --security-audit
```

```
# Review access logs
```

```
grep -E "(401|403|429)" /var/log/sentinel/access.log | tail -50
```

```
```
```

### High Priority Alert Response (P2)

**Trigger:** Performance degradation, high error rates, authentication issues

### Response Procedure:

1. Validate alert accuracy with manual health check
2. Check system resources: `htop`, `df -h`, `free -m`
3. Review application metrics:

```
bash curl -s http://localhost:8080/api/metrics/snapshot | jq '.'
```

4. Check for recent configuration changes
5. Document findings and create incident report

## Medium Priority Alert Response (P3)

**Trigger:** Anomaly detection, suspicious activity, configuration drift

### Response Procedure:

1. Review anomaly details:

```
bash curl -s http://localhost:8080/api/anomaly/score | jq '.'
```

2. Check recent detections and alerts
3. Validate configuration integrity:

```
bash python tools/verify_minimax_claims.py --config-check
```

4. Schedule detailed investigation during maintenance window

## 3. Incident Response Playbooks

### 3.1 Security Incident Response

#### Phase 1: Detection and Analysis

```
Immediate security assessment
python tools/sec_lint.py --incident-scan --output
incident_scan.json

Check for indicators of compromise
grep -E "(admin|root|su|sudo)" /var/log/sentinel/app.log | tail -20

Review authentication logs
grep -E "(login|auth|token)" /var/log/sentinel/access.log | tail
-50

Network connection analysis
netstat -tulnp | grep :8080
```

## Phase 2: Containment

```
Enable enhanced logging
export SENTINEL_DEBUG_MODE=1
sudo systemctl reload sentinel_service_runner

Block suspicious IPs (example)
sudo ufw insert 1 deny from 192.168.1.100 comment "Security
incident containment"

Rotate admin tokens
python tools/rotate_secrets.py --admin-tokens-only --force

Enable audit mode
python tools/api_contract_check.py --audit-mode --continuous
```

## Phase 3: Eradication and Recovery

```
Clean up suspicious files
python tools/quarantine.py --scan-and-quarantine

Restore from clean backup if necessary
python tools/backup_restore.py --restore --backup-id
clean_backup_id

Update signatures and rules
python tools/update_threat_intel.py --force-update

Validate system integrity
python tools/verify_minimax_claims.py --full-verification
```

## Phase 4: Post-Incident Activities

```
Generate incident report
python tools/generate_incident_report.py --incident-id IR-$(date +
%Y%m%d-%H%M)

Update security baselines
python tools/sec_lint.py --update-baseline

Review and update security policies
python tools/policy_update.py --security-hardening
```

## 3.2 Performance Incident Response

### Performance Degradation Response

```
Quick performance assessment
python tools/microbench.py --quick-diagnostic

Check system resources
top -b -n 1 | head -20
iostat 1 5
vmstat 1 5

Database/storage analysis
du -sh /opt/sentinel/data/*
df -i # Check inode usage

Network performance check
ss -tuln | grep :8080
curl -w "@curl-format.txt" -o /dev/null -s http://localhost:8080/
health
```

## Memory/Resource Issues

```
Memory analysis
free -m
cat /proc/meminfo
pmap $(pgrep -f sentinel)

Check for memory leaks
python tools/memory_profiler.py --profile-duration 300

Restart with memory monitoring
sudo systemctl stop sentinel_service_runner
sudo systemctl start sentinel_service_runner
python tools/monitor_resource_usage.py --duration 3600
```

## **4. Maintenance & Updates Playbooks**

### **4.1 Regular Maintenance Tasks**

#### **Daily Maintenance Checklist**

```
#!/bin/bash
daily_maintenance.sh

echo "=== Daily Sentinel Maintenance ==="

Check service health
echo "1. Service health check:"
systemctl is-active sentinel_service_runner && echo "✓ Service running" || echo "x Service down"

Check disk space
echo "2. Disk space check:"
df -h /opt/sentinel | awk 'NR==2 {print "Usage: " $5}'

Check log file sizes
echo "3. Log file check:"
du -sh /var/log/sentinel/* | sort -h

Backup verification
echo "4. Backup verification:"
python tools/backup_restore.py --verify-latest

Security scan
echo "5. Security scan:"
python tools/sec_lint.py --daily-scan --brief

Performance check
echo "6. Performance check:"
python tools/microbench.py --quick --compare-baseline

echo "=== Maintenance Complete ==="
```

## Weekly Maintenance Tasks

```
#!/bin/bash
weekly_maintenance.sh

echo "=== Weekly Sentinel Maintenance ==="

Full system backup
echo "1. Creating weekly backup:"
python tools/backup_restore.py --full-backup --label "weekly-$
(date +%Y%m%d)"

Log rotation and cleanup
echo "2. Log maintenance:"
sudo logrotate -f /etc/logrotate.d/sentinel
find /var/log/sentinel -name "*.gz" -mtime +30 -delete

Security updates
echo "3. Security update check:"
python tools/sec_lint.py --update-signatures
python tools/threat_intel_update.py --weekly-update

Performance baseline update
echo "4. Performance baseline update:"
python tools/microbench.py --update-baseline

Configuration validation
echo "5. Configuration validation:"
python tools/verify_minimax_claims.py --weekly-check

echo "=== Weekly Maintenance Complete ==="
```

## **4.2 Update and Upgrade Procedures**

### **Application Update Procedure**

```

#!/bin/bash
update_sentinel.sh

NEW_VERSION="$1"
if [-z "$NEW_VERSION"]; then
 echo "Usage: $0 <new_version>"
 exit 1
fi

echo "=== Sentinel Update to $NEW_VERSION ==="

Pre-update backup
echo "1. Creating pre-update backup:"
python tools/backup_restore.py --pre-update-backup --version
"$NEW_VERSION"

Health check before update
echo "2. Pre-update health check:"
python tools/composite_health_check.py --pre-update

Download new version
echo "3. Downloading new version:"
wget "https://releases.watchlockai.com/sentinel-${NEW_VERSION}.zip"

Verify checksum
echo "4. Verifying download:"
sha256sum -c "sentinel-${NEW_VERSION}.sha256"

Stop service
echo "5. Stopping service:"
sudo systemctl stop sentinel_service_runner

Backup current version
echo "6. Backing up current version:"
sudo cp -r /opt/sentinel/app "/opt/sentinel/app.backup.${date +

```

```
%Y%m%d)"

Install new version
echo "7. Installing new version:"
cd /opt/sentinel/app
sudo -u sentinel-service unzip "sentinel-${NEW_VERSION}.zip"

Update configuration if needed
echo "8. Configuration update:"
python tools/config_migrate.py --from-backup "/opt/sentinel/
app.backup.$(date +%Y%m%d)"

Start service
echo "9. Starting service:"
sudo systemctl start sentinel_service_runner

Post-update validation
echo "10. Post-update validation:"
sleep 30 # Allow service to start
python tools/composite_health_check.py --post-update
python tools/microbench.py --validate-performance

echo "=== Update Complete ==="
```

## Rollback Procedure

```
#!/bin/bash
rollback_sentinel.sh

BACKUP_DATE="$1"
if [-z "$BACKUP_DATE"]; then
 echo "Usage: $0 <backup_date_YYYYMMDD>"
 exit 1
fi

echo "=== Sentinel Rollback to $BACKUP_DATE ==="

Stop current service
sudo systemctl stop sentinel_service_runner

Restore from backup
sudo rm -rf /opt/sentinel/app
sudo cp -r "/opt/sentinel/app.backup.$BACKUP_DATE" /opt/sentinel/app
sudo chown -R sentinel-service:sentinel-service /opt/sentinel/app

Start service
sudo systemctl start sentinel_service_runner

Validate rollback
sleep 30
python tools/composite_health_check.py --rollback-validation

echo "=== Rollback Complete ==="
```

# 5. Troubleshooting Playbooks

## 5.1 Common Issue Diagnostics

### Service Won't Start

```
Check service status
sudo systemctl status sentinel_service_runner

Check logs for errors
sudo journalctl -u sentinel_service_runner --since "10 minutes ago"

Check application logs
tail -50 /var/log/sentinel/app.log | grep -E "(ERROR|CRITICAL|FATAL)"

Check configuration
python tools/config_validate.py --check-syntax

Check permissions
ls -la /opt/sentinel/app/
ls -la /opt/sentinel/config/

Check port availability
sudo netstat -tulnp | grep :8080
```

## High CPU Usage

```
Identify process causing high CPU
top -b -n 1 | grep sentinel

Check for infinite loops or high-frequency operations
python tools/perf_probe.py --cpu-analysis

Review recent configuration changes
git log --oneline --since="24 hours ago" config/

Check for memory leaks affecting CPU
python tools/memory_profiler.py --cpu-correlation
```

## Memory Issues

```
Check current memory usage
free -m
ps aux | grep sentinel | awk '{print $6}' | head -1 # RSS in KB

Memory trend analysis
python tools/memory_trend_analysis.py --last-24h

Check for memory leaks
valgrind --tool=memcheck --leak-check=full python app.py

Garbage collection analysis
python -c "import gc; gc.set_debug(gc.DEBUG_STATS);
exec(open('app.py').read())"
```

## Network Connectivity Issues

```
Check listening ports
sudo netstat -tulnp | grep sentinel

Test local connectivity
curl -v http://localhost:8080/health

Check firewall rules
sudo ufw status verbose

Test external connectivity
curl -v http://external-ip:8080/health

Check DNS resolution
nslookup sentinel.example.com
```

## 5.2 Advanced Troubleshooting

### Performance Debugging

```
Comprehensive performance analysis
python tools/microbench.py --debug-mode --detailed

System call tracing
sudo strace -p $(pgrep -f sentinel) -o strace_output.txt &
sleep 60
sudo kill %1

I/O analysis
sudo iotop -p $(pgrep -f sentinel)
sudo iostat -x 1 10

Network tracing
sudo tcpdump -i any -n port 8080 -w network_trace.pcap
```

## Security Debugging

```
Comprehensive security analysis
python tools/sec_lint.py --debug-mode --verbose

Access log analysis
awk '{print $1}' /var/log/sentinel/access.log | sort | uniq -c |
sort -nr | head -20

Authentication debugging
grep -E "(auth|login|token)" /var/log/sentinel/app.log | tail -50

Permission analysis
sudo find /opt/sentinel -type f -perm /o+w # World-writable files
sudo find /opt/sentinel -type f -perm /u+s # SUID files
```

## **6. Security Operations Playbooks**

### **6.1 Security Monitoring**

#### **Real-time Security Monitoring**

```
#!/bin/bash
security_monitor.sh

echo "=== Sentinel Security Monitoring ==="

Check for failed authentication attempts
echo "Failed auth attempts in last hour:"
grep -E "(401|403)" /var/log/sentinel/access.log | \
 awk -v cutoff=<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mo stretchy="false">(</mo><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mo>−</mo><msup><mi>d</
mi><mi>′</mi></msup><mn>1</mn><mi>h</mi><mi>o</mi><mi>u</
mi><mi>r</mi><mi>a</mi><mi>g</mi><msup><mi>o</mi><mi>″</
mi></msup><mo>+</mo></mrow></math>4 > cutoff' | wc -
l

Check for admin access
echo "Admin endpoint access in last hour:"
grep "/api/admin" /var/log/sentinel/access.log | \
 awk -v cutoff=<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mo stretchy="false">(</mo><mi>d</
mi><mi>a</mi><mi>t</mi><mi>e</mi><mo>−</mo><msup><mi>d</
mi><mi>′</mi></msup><mn>1</mn><mi>h</mi><mi>o</mi><mi>u</
mi><mi>r</mi><mi>a</mi><mi>g</mi><msup><mi>o</mi><mi>″</
mi></msup><mo>+</mo></mrow></math>4 > cutoff' | wc -
l

Anomaly score check
echo "Current anomaly score:"
curl -s http://localhost:8080/api/anomaly/score | jq '.score'

Recent security events
```

```
echo "Recent security events:"
python tools/security_event_analyzer.py --last-hour --summary
```

## Weekly Security Review

```
#!/bin/bash
weekly_security_review.sh

echo "=== Weekly Security Review ==="

Comprehensive security scan
python tools/sec_lint.py --comprehensive --output
weekly_security_report.json

Access pattern analysis
echo "Top IP addresses accessing the service:"
awk '{print $1}' /var/log/sentinel/access.log | sort | uniq -c |
sort -nr | head -10

Authentication analysis
echo "Authentication statistics:"
python tools/auth_analyzer.py --weekly-stats

Configuration drift detection
echo "Configuration changes:"
python tools/config_drift_detector.py --weekly-check

Security patch status
echo "Security update status:"
python tools/security_updater.py --check-updates --report-only
```

## 6.2 Incident Response Automation

### Automated Threat Response

```
#!/bin/bash
automated_threat_response.sh

THREAT_SCORE=$(curl -s http://localhost:8080/api/anomaly/score |
jq -r '.score')
THRESHOLD=0.8

if (($>$
xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mo stretchy="false">(</mo><mi>e</
mi><mi>c</mi><mi>h</mi><mi>o</mi><mi></mi></mrow></math></
span>THREAT_SCORE > $THRESHOLD" | bc -l))); then
 echo "High threat score detected: $THREAT_SCORE"

 # Enable enhanced monitoring
 python tools/monitoring_enhancer.py --threat-mode

 # Collect forensic data
 python tools/forensic_collector.py --automated --output
 "threat_$(date +%s)"

 # Alert security team
 echo "Automated threat response triggered" | \
 mail -s "SECURITY ALERT: High Threat Score" security-
 team@example.com

 # Optional: Enable protective mode
 # python tools/protective_mode.py --enable --duration 3600
fi
```

## **7. Backup and Recovery Playbooks**

### **7.1 Backup Procedures**

#### **Daily Backup Script**

```

#!/bin/bash
daily_backup.sh

BACKUP_DIR="/opt/sentinel/backups"
DATE=$(date +%Y%m%d_%H%M%S)
RETENTION_DAYS=30

echo "=== Daily Backup Process ==="

Create backup directory
mkdir -p "$BACKUP_DIR/daily"

Application backup
tar -czf "BACKUP_DIR/daily/app_DATE.tar.gz" -C /opt/sentinel app/

Configuration backup
tar -czf "BACKUP_DIR/daily/config_DATE.tar.gz" -C /opt/sentinel config/

Data backup (if applicable)
if [-d "/opt/sentinel/data"]; then
 tar -czf "BACKUP_DIR/daily/data_DATE.tar.gz" -C /opt/sentinel data/
fi

Database backup (if applicable)
python tools/backup_restore.py --database-backup --output "BACKUP_DIR/daily/db_DATE.sql"

Cleanup old backups

```

```

find "<math
xmlns='http://www.w3.org/1998/Math/MathML'
display='inline'"><mrow><mi>B</mi><mi>A</mi><mi>C</mi><mi>K</
mi><mi>U</mi><msub><mi>P</mi><mi>D</mi></msub><mi>I</mi><mi>R</
mi><mo>/</mo><mi>d</mi><mi>a</mi><mi>i</mi><mi>l</
mi><mi>y</mi><mi>"</mi><mo>−</mo><mi>n</mi><mi>a</
mi><mi>m</mi><mi>e</mi><mi>"</mi><mo>*</mo><mo>.</
mo><mi>t</mi><mi>a</mi><mi>r</mi><mo>.</mo><mi>g</
mi><mi>z</mi><mi>"</mi><mo>−</mo><mi>m</mi><mi>t</
mi><mi>i</mi><mi>m</mi><mi>e</mi><mo>+</mo></mrow></math></
span>RETENTION_DAYS -delete
find "<math
xmlns='http://www.w3.org/1998/Math/MathML'
display='inline'"><mrow><mi>B</mi><mi>A</mi><mi>C</mi><mi>K</
mi><mi>U</mi><msub><mi>P</mi><mi>D</mi></msub><mi>I</mi><mi>R</
mi><mo>/</mo><mi>d</mi><mi>a</mi><mi>i</mi><mi>l</
mi><mi>y</mi><mi>"</mi><mo>−</mo><mi>n</mi><mi>a</
mi><mi>m</mi><mi>e</mi><mi>"</mi><mo>*</mo><mo>.</
mo><mi>s</mi><mi>q</mi><mi>l</mi><mi>"</mi><mo>−</
mo><mi>m</mi><mi>t</mi><mi>i</mi><mi>m</mi><mi>e</
mi><mo>+</mo></mrow></math>RETENTION_DAYS -delete

echo "Backup completed: <span class='math-inline' style='font-
family: serif;'">BACKUP_DIR/daily/*_DATE.*"

```

## **7.2 Recovery Procedures**

### **Disaster Recovery Process**

```

#!/bin/bash
disaster_recovery.sh

BACKUP_DATE="$1"
if [-z "$BACKUP_DATE"]; then
 echo "Usage: $0 <backup_date_YYYYMMDD_HHMMSS>"
 exit 1
fi

echo "=== Disaster Recovery Process ==="

Stop services
sudo systemctl stop sentinel_service_runner

Restore application
cd /opt/sentinel
sudo rm -rf app/
sudo tar -xzf "backups/daily/app_${BACKUP_DATE}.tar.gz"

Restore configuration
sudo rm -rf config/
sudo tar -xzf "backups/daily/config_${BACKUP_DATE}.tar.gz"

Restore data (if applicable)
if [-f "backups/daily/data_${BACKUP_DATE}.tar.gz"]; then
 sudo rm -rf data/
 sudo tar -xzf "backups/daily/data_${BACKUP_DATE}.tar.gz"
fi

Restore database (if applicable)
if [-f "backups/daily/db_${BACKUP_DATE}.sql"]; then
 python tools/backup_restore.py --database-restore --input
 "backups/daily/db_${BACKUP_DATE}.sql"
fi

```

```
Fix permissions
sudo chown -R sentinel-service:sentinel-service /opt/sentinel

Start services
sudo systemctl start sentinel_service_runner

Validate recovery
sleep 30
python tools/composite_health_check.py --disaster-recovery-
validation

echo "=== Disaster Recovery Complete ==="
```

## **8. Automation and Integration**

### **8.1 CI/CD Integration**

#### **Automated Testing Pipeline**

```

.github/workflows/sentinel-ops.yml
name: Sentinel Operations Pipeline

on:
 push:
 branches: [main]
 schedule:
 - cron: '0 2 * * *' # Daily at 2 AM

jobs:
 operational-tests:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2

 - name: Setup Python
 uses: actions/setup-python@v2
 with:
 python-version: '3.12'

 - name: Install dependencies
 run: pip install -r requirements.txt

 - name: Security scan
 run: python tools/sec_lint.py --ci-mode

 - name: Performance baseline
 run: python tools/microbench.py --ci-mode

 - name: Configuration validation
 run: python tools/verify_minimax_claims.py --ci-check

 - name: Backup test
 run: python tools/backup_restore.py --test-mode

```

## 8.2 Monitoring Integration

### Nagios Integration

```
#!/bin/bash
nagios_sentinel_check.sh

Nagios plugin for Sentinel monitoring
Usage: check_sentinel_health.sh -H hostname -p port

while getopts "H:p:" opt; do
 case $opt in
 H) HOSTNAME="$OPTARG";;
 p) PORT="$OPTARG";;
 esac
done

HOSTNAME=${HOSTNAME:-localhost}
PORT=${PORT:-8080}

Health check
if curl -sf "http://<span class="math-inline" style="display:
inline;"><math xmlns="http://www.w3.org/1998/Math/MathML"
display="inline"><mrow><mi>H</mi><mi>O</mi><mi>S</mi><mi>T</
mi><mi>N</mi><mi>A</mi><mi>M</mi><mi>E</mi><mi>:</mi></mrow></
math>PORT/health" > /dev/null; then
 echo "OK - Sentinel service healthy"
 exit 0
else
 echo "CRITICAL - Sentinel service health check failed"
 exit 2
fi
```

## Zabbix Integration

```
{
 "zabbix_export": {
 "templates": [
 {
 "template": "Template Sentinel Service",
 "items": [
 {
 "name": "Sentinel Health Status",
 "key": "sentinel.health",
 "type": "HTTP agent",
 "url": "http://localhost:8080/health",
 "value_type": "Numeric (unsigned)"
 }
],
 "triggers": [
 {
 "expression": "{Template Sentinel Service:sentinel.health.last()}=0",
 "name": "Sentinel service is down",
 "priority": "High"
 }
]
 }
]
 }
}
```

## 9. Documentation and Training

### 9.1 Runbook Quick Reference

#### Emergency Contacts

- **Incident Commander:** +1-555-0101
- **Security Team:** security-oncall@example.com
- **Engineering Team:** eng-oncall@example.com
- **Management Escalation:** management@example.com

## Critical Commands Quick Reference

```
Service control
sudo systemctl {start|stop|restart|status} sentinel_service_runner

Health check
curl -f http://localhost:8080/health

Emergency shutdown
sudo systemctl stop sentinel_service_runner && sudo pkill -f
sentinel

Emergency security lockdown
python tools/emergency_lockdown.py --enable

View recent logs
sudo journalctl -u sentinel_service_runner --since "1 hour ago"

Check disk space
df -h /opt/sentinel

Manual backup
python tools/backup_restore.py --emergency-backup
```

## 9.2 Training Materials

### New Team Member Onboarding Checklist

#### 1. Access Setup

- ☐ Server access (SSH keys)
- ☐ Admin credentials
- ☐ Monitoring system access
- ☐ Documentation access

## 2. Tool Familiarization

- [ ] Complete SDK tutorial: `python tools/sentinel_sdk.py --tutorial`
- [ ] Run health checks: `python tools/composite_health_check.py --demo`
- [ ] Practice backup/restore: `python tools/backup_restore.py --training-mode`

## 3. Incident Response Training

- [ ] Shadow experienced team member during incident
- [ ] Practice runbook procedures in test environment
- [ ] Complete security incident simulation

## 4. Certification

- [ ] Pass operational knowledge assessment
- [ ] Demonstrate incident response capabilities
- [ ] Complete hands-on troubleshooting test

# Conclusion

These operational playbooks provide comprehensive guidance for production operation of WatchLockAI Sentinel. The procedures are designed to ensure reliable, secure, and performant operation while providing clear escalation paths and incident response protocols.

### Key Operational Areas Covered:

- **Deployment & Installation:** Production-ready deployment procedures
- **Monitoring & Alerting:** Comprehensive monitoring and alert response
- **Incident Response:** Security and performance incident handling
- **Maintenance & Updates:** Regular maintenance and upgrade procedures
- **Troubleshooting:** Diagnostic and resolution procedures
- **Security Operations:** Security-focused operational procedures

Regular review and updates of these playbooks ensure they remain current with system changes and operational requirements.

---

**Playbooks Generated By:** MiniMax Agent

**Operational Framework:** Production-ready procedures for WatchLockAI Sentinel

**Verification:** These playbooks provide complete P12 operational guidance with comprehensive procedures for all operational scenarios.